

Project 2: Sentiment Analysis on Movie Reviews

- **The Goal:** Build an AI that reads a movie review (text) and determines if the reviewer liked the movie (Positive) or hated it (Negative).
- **The Challenge:** Text is messy. Unlike images (fixed 28x28 grids), sentences have different lengths, slang, and context. A "Dense" network fails here because order matters ("*not good*" is the opposite of "*good*").
- **The Solution: Recurrent Neural Networks (RNNs).** These networks have "memory." They read word-by-word, remembering what they just read.

Step 1: The "Text-to-Number" Problem

Computers cannot read English. They only understand numbers.

Before we build the brain, we must translate the language.

1. **Tokenization:** We give every word a unique ID number.
 - a. "The" = 1
 - b. "Movie" = 23
 - c. "Bad" = 405
2. **Padding:** Since neural networks need fixed-size inputs, we make every review the same length (e.g., 256 words). If a review is short, we add zeros (padding). If it's long, we chop it off.

Step 2: The Code (IMDB Dataset)

We will use the **IMDB Dataset** built into Keras. It contains 25,000 reviews, already converted to numbers for us (to save time cleaning text).

Copy this block into a **new Colab cell**:

```
Python
import tensorflow as tf
from tensorflow.keras import layers
import numpy as np

# 1. Load the Data
# num_words=10000: We only keep the 10,000 most common words. Rare words are
# ignored.
print("Loading data...")
(x_train, y_train), (x_test, y_test) =
tf.keras.datasets.imdb.load_data(num_words=10000)

# 2. Preprocessing (Padding)
```

```

# We force every review to be exactly 256 words long.
x_train = tf.keras.preprocessing.sequence.pad_sequences(x_train, maxlen=256)
x_test = tf.keras.preprocessing.sequence.pad_sequences(x_test, maxlen=256)

print(f"Training data shape: {x_train.shape}")
print(f"A single review (in numbers): {x_train[0][:10]} ...")

# 3. Build the RNN Model
model = tf.keras.Sequential([
    # Layer 1: Embedding
    # Turns positive integers (indexes) into dense vectors of fixed size.
    # Imagine a 3D map where "Good" and "Great" are close, but "Bad" is far
    away.
    layers.Embedding(input_dim=10000, output_dim=32),

    # Layer 2: SimpleRNN
    # Reads the input sequence step-by-step.
    # 32 units = The size of its "memory".
    layers.SimpleRNN(32),

    # Layer 3: Dense (Output)
    # Single neuron. If output > 0.5, it's Positive. If < 0.5, it's Negative.
    layers.Dense(1, activation='sigmoid')
])

model.compile(optimizer='adam',
              loss='binary_crossentropy', # Used for Yes/No classification
              metrics=['accuracy'])

# 4. Train
print("Training model...")
history = model.fit(x_train, y_train, epochs=5, batch_size=128,
                    validation_split=0.2)

# 5. Evaluate
results = model.evaluate(x_test, y_test)
print(f"Test Accuracy: {results[1]*100:.2f}%")

```

Step 3: Understanding the "Magic" Layers

While that runs (it might take 1-2 minutes), let's break down the two new concepts introduced here.

1. The Embedding Layer (*layers.Embedding*)

In the digit project, 7 meant the number 7.

In language, 7 is just the ID for a word (maybe "apple"). The number 7 has no math relationship to 8 ("orange").

- **The Problem:** If we just feed ID numbers, the model might think word #8 is "bigger" than word #7.
- **The Solution:** The Embedding layer learns a **Vector** for each word.
 - It turns "King" into [0.9, 0.2, 0.5]
 - It turns "Queen" into [0.9, 0.3, 0.5]
 - It turns "Apple" into [0.1, 0.9, 0.1]
 - The model *learns* these numbers during training so that similar words cluster together in mathematical space.

2. The SimpleRNN Layer (*layers.SimpleRNN*)

- **The Logic:** A normal neuron takes input, gives output, and forgets.
- **The RNN Neuron:** It takes input, gives output, and **passes a copy of that output to itself** for the next word.
 - When it reads the word "not", it creates a "negative context" state.
 - When it reads "good" next, that "negative context" flips the meaning of "good" to bad.
 - This "Loop" allows it to understand sequences.

Step 4: Analyze Your Results

You will likely see an accuracy around **80% to 85%**.

Why not 98% like the digits?

1. **Complexity:** Language is much harder than handwriting. Sarcasm ("Oh, great movie... NOT") confuses simple models.
2. **Vanishing Gradient:** SimpleRNN has a short memory. If the review is long, it "forgets" the beginning of the sentence by the time it reaches the end.

The Fix:

In advanced NLP, we replace SimpleRNN with LSTM (Long Short-Term Memory) or GRU. They are designed to remember things for a long time.

Task for you:

Once the code finishes, try changing `layers.SimpleRNN(32)` to `layers.LSTM(32)` and run it again. Does the accuracy improve?

1. The SimpleRNN (The Goldfish) It passes information in a loop, but as the loop spins, the information degrades. Mathematical operations (multiplication) eventually dilate or shrink the numbers until the original meaning is lost. It has a **Short-Term Memory**.

2. The LSTM (The Notepad) LSTM stands for **Long Short-Term Memory**. It adds a physical "highway" for information to flow through unchanged.

Inside an LSTM cell, there are three "Gates" (Filters) that control this highway:

- **The Forget Gate:** "Should I delete this old info?" (e.g., The sentence changed from talking about the plot to talking about the actors -> Forget the plot).
- **The Input Gate:** "Is this new info important?"
- **The Output Gate:** "What part of my memory should I reveal right now?"

By selectively remembering and forgetting, it can connect the word "not" at the start of a sentence to the word "recommend" at the end, realizing the review is negative.

Understanding of Code:

Phase 1: Data Engineering

Before we can learn, we must organize.

Python

```
(x_train, y_train), (x_test, y_test) =  
tf.keras.datasets.imdb.load_data(num_words=10000)
```

- **tf.keras.datasets.imdb:** Keras has the IMDB dataset built-in.
- **num_words=10000:** This is a filter. The English language has hundreds of thousands of words. Most are rare (e.g., "perspicacious"). Rare words confuse the model because they don't appear enough to learn a pattern.
 - **The Logic:** We strictly keep the **top 10,000 most frequently used words**.
 - **The Effect:** Any word ranking 10,001 or lower is discarded and replaced with a special "Unknown" token.

Python

```
x_train = tf.keras.preprocessing.sequence.pad_sequences(x_train, maxlen=256)  
x_test = tf.keras.preprocessing.sequence.pad_sequences(x_test, maxlen=256)
```

- **pad_sequences:** Neural networks require **tensor uniformity**. They cannot accept input row 1 having length 50 and input row 2 having length 100. It breaks the matrix multiplication.
- **maxlen=256:** We set a strict rule—every review must be exactly 256 words long.

- **Case A (Short Review):** A 50-word review gets 206 zeros added to the front (Padding).
- **Case B (Long Review):** A 500-word review gets the last 244 words chopped off (Truncation).
- **Result:** `x_train` is now a perfect rectangle (matrix) of size (25000, 256).

Phase 2: Architecture Design (The Brain)

This is where we define how the machine thinks.

Python

```
model = tf.keras.Sequential([ ... ])
```

- **Sequential:** Creates a linear stack of layers. Data flows from top to bottom.

Layer 1: The Translator

Python

```
layers.Embedding(input_dim=10000, output_dim=32)
```

- **input_dim=10000:** We tell the layer to expect integers between 0 and 9,999 (our vocabulary size).
- **output_dim=32:** We tell the layer to represent every single word as a vector of **32 numbers**.
- **The Logic (Word Embeddings):**
 - Initially, the word "Love" (ID: 54) is just a random list of 32 numbers.
 - During training, the model adjusts these numbers.
 - Eventually, the math for "Love" will look very similar to "Like" and very different from "Hate".
 - It turns **Integers (Symbols)** into **Geometric Positions (Meaning)**.

Layer 2: The Memory

Python

```
layers.SimpleRNN(32)
```

- **SimpleRNN:** A Recurrent Neural Network layer.
- **32:** The size of the "hidden state" (memory). It remembers 32 features of the sentence at any given time.

- **The Logic (The Loop):**
 - It reads the first word vector.
 - It produces an output AND updates its internal "state" (memory).
 - It reads the second word **combined** with the memory of the first word.
 - It repeats this 256 times (for the sequence length).
 - After the last word, it passes the *final* accumulated meaning to the next layer.

Layer 3: The Judge

Python

```
layers.Dense(1, activation='sigmoid')
```

- **Dense(1):** We only need **one neuron**. Why? Because the answer is binary.
 - 0 = Negative
 - 1 = Positive
- **activation='sigmoid':**
 - In the digit project, we used softmax (for 10 classes).
 - Here, we use **Sigmoid**. It forces the raw output number to be strictly between **0.0 and 1.0**.
 - If the output is 0.9, the model is 90% sure it's Positive.

Phase 3: Execution (Compiling & Training)

Python

```
model.compile(optimizer='adam',
              loss='binary_crossentropy',
              metrics=['accuracy'])
```

- **optimizer='adam':** The same adaptive mechanic we used before. It tweaks the weights to minimize error.
- **loss='binary_crossentropy':**
 - "Binary" because we have 2 classes.
 - "Crossentropy" measures the distance between the predicted probability (e.g., 0.8) and the actual label (e.g., 1.0).
 - Standard math for Yes/No problems.

Python

```
history = model.fit(x_train, y_train, epochs=5, batch_size=128,
                  validation_split=0.2)
```

- **batch_size=128:** The model doesn't learn from one review at a time (too slow), nor all 25,000 (too big for RAM). It grabs **128 reviews**, calculates the average error, and updates the weights once. This happens repeatedly until all reviews are seen.
- **validation_split=0.2:** This is crucial.
 - It takes 20% of x_train (5,000 reviews) and **locks them away**.
 - The model **never** trains on these.
 - At the end of every epoch, it tests itself on these 5,000 reviews.
 - **Why?** To check for **Overfitting**. If Training Accuracy goes up (99%) but Validation Accuracy stays low (80%), the student is memorizing, not learning.

Python

```
results = model.evaluate(x_test, y_test)
```

- **evaluate:** The final exam. It runs the model on the completely unseen x_test data to give you the real-world accuracy.

Summary of the Flow

1. **Input:** A list of 256 Integers (Word IDs).
2. **Embedding:** Converts IDs to meaningful vectors (Lists of 32 numbers).
3. **RNN:** Reads the vectors one by one, updating its "memory" of the sentence context.
4. **Dense:** Takes the final memory state and calculates a "Positivity Score."
5. **Sigmoid:** Squashes that score between 0 and 1.

Does this breakdown help you visualize the data flowing through the "pipes" of the network?